

UNIVERSIDAD TECNOLÓGICA NACIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

MATERIA:

Programación 1

TRABAJO PRÁCTICO:

Trabajo Práctico Integrador (TPI): Gestión de Datos de Países en Python *Filtros, ordenamientos y estadísticas*

INTEGRANTES:

Estudiante: Matias Ferreyra

FECHA DE ENTREGA:

1 de junio de 2026

INDICE

2	-----	Índice
3	-----	Marco teórico y explicaciones lógicas
4	-----	Marco teórico y explicaciones lógicas
5	-----	Diagrama de Flujo
6	-----	Simulación código, identificación errores
7	-----	Simulación código, identificación errores
8	-----	Simulación código, identificación errores
9	-----	Reflexión Final
10	-----	Enlaces y Bibliografías

CONCEPTOS APLICADOS A LO LARGO DEL PROYECTO

Diccionarios: Son arreglos no ordenados cuyo tipo de organización es ‘clave-valor’. Los datos se almacenan mediante ‘keys’ de manera asociativa. En el código se utiliza en el diccionario ‘paisCont’, donde se va anexando el nombre del continente como ‘key’, y el número de veces que aparece por país, como ‘value’

Listas: Son estructuras que permiten almacenar datos en Python. Cuyas características principales son:

- Mutables: Sus valores y extensión pueden cambiar a lo largo del programa
- Heterogéneas: Pueden almacenar muchos tipos de valores
- Ordenadas: Cada elemento tiene su posición asignada(índice) en la lista
- Duplicados: Permite almacenar datos duplicados dentro de la lista

A lo largo del código se utilizan muchas listas para almacenar datos. De las más importantes tenemos ‘datos’, donde se almacenan todos los valores leídos del archivo .csv

Funciones: Son fracciones de código reutilizables y organizados, que ejecutan un bloque de instrucciones específicas y devuelven un valor. En el código se usan múltiples funciones para evitar repetir código, y además brindar una mayor legibilidad. La más importante ‘traerDatos()’, es el encargado de leer y mapear los datos en el archivo ‘.csv’

Condicionales: Son estructuras de control, que permiten al código tomar decisiones en tiempo de ejecución. El bloque de código ‘if’ se ejecuta cuando una condición

especifica se cumple. 'elif' se ejecuta cuando se cumple otra condición apartada de la primera. 'else' se cumple cuando ninguna condición anterior es verdadera.

Estructuras repetitivas: Son estructuras que ejecutan un bloque de código repetidamente. El periodo de tiempo en el que se ejecuta depende del tipo de 'loop'

- **For loop:** se ejecuta un numero definido de veces, usado en el código usualmente para iterar un numero fijo de veces, por ejemplo, iterar por cada dato del arreglo 'datos', la cantidad de veces que se itera en la extensión del arreglo `datos(len(datos))`
- **While loop:** Se ejecuta siempre y cuando la condición que abarca sea verdadera. Usada, por ejemplo, en el menú principal, mientras el usuario no elija la opción 'salir', el bucle del menú no terminara

En el código hay muchos condicionales, ya que es una pieza clave para el funcionamiento de este. Uno de los principales, es el condicional que verifica cual es la 'opcion' insertada por el usuario en el menú principal del programa. Corroborando si algunas de las condiciones se cumplen

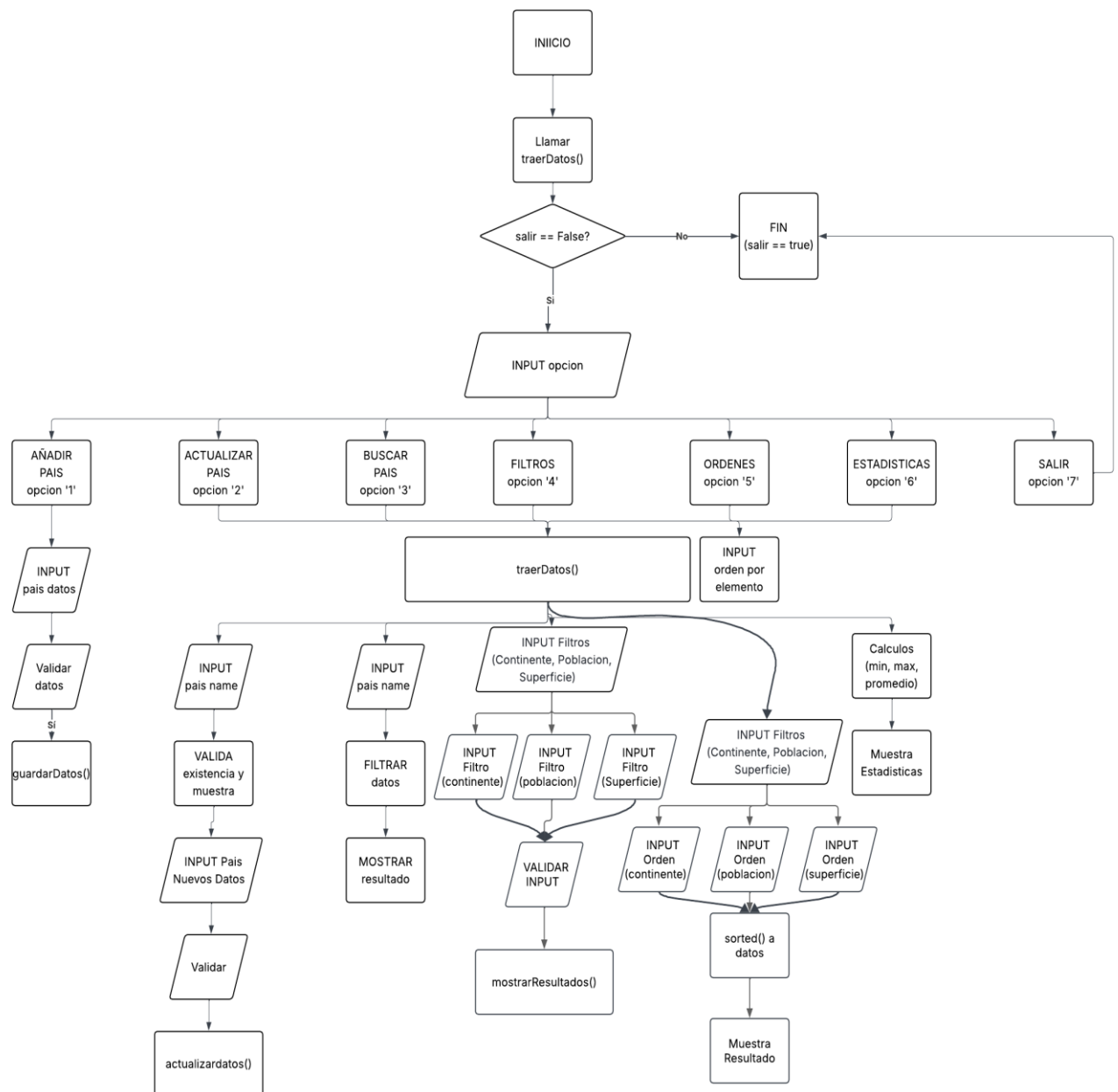
Ordenamientos: Algoritmos diseñados para ordenar los elementos de una colección, en base a un criterio o formato. Casi al final del código, se utiliza `sorted()`, una función que permite organizar los elementos del arreglo principal (datos) de forma ascendente, indicándole que elemento de este es el que se debe realizar el ordenamiento

Estadísticas básicas: Al final el programa, para cumplir con la opción requerida del sistema 'mostrar estadísticas', se realizan una serie de cálculos para obtener los datos solicitados. Un ejemplo de esto es 'mayorPoblacion' donde se utiliza la función '`max()`' para determinar el resultado

Archivo CSV: Es un archivo de texto plano, utilizado para el almacenamiento de datos. Cada fila representa un elemento, cada elemento está separado por comas. En el archivo .csv del proyecto, se encuentran almacenados los países con sus datos. Cuenta con 4 columnas:

1. Nombre del país
2. Población del país
3. Superficie del país
4. Nombre del Continente

DIAGRAMA LOGICO DEL CODIGO PYTHON



SIMULACION DE ALGUNAS DE LAS FUNCIONES DEL CODIGO

AGREGAR PAIS

- Valores iniciales del archivo .csv

```
models > TplIntegrador > archivo.csv > data
1  Argentina,45376763,2780400,América
2  Japón,125800000,377975,Asia
3  Brasil,213993437,8515767,América
4  Alemania,344344,33333,Asia
5  Asas,2,2,Asia
6  Asa,2121212,2222222,Europa
7
```

- Como vemos, si intentamos añadir un país que ya está registrado en el archivo .csv, el sistema devuelve un mensaje de error

```
models > TplIntegrador > TrabajoIntegradorProgramacion.py > ...
1  #TRABAJO INTEGRADOR PROGRAMACION
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
y"
==BIENVENIDO AL DICCIONARIO DE PAISES==

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadisticas
7) Salir
Que opcion desea elegir: 1

==AÑADIR PAIS==
Dame el nombre del pais: Argentina
Nombre de pais no válido
```

- Por otro lado, si se ingresa un valor que no es un entero, o es un entero menor a 0, se devuelve un mensaje de error

```
==AÑADIR PAIS==
Dame el nombre del pais: Mexico
De cuanto es la poblacion de Mexico: g
Poblacion no válida

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadisticas
7) Salir
Que opcion desea elegir: 
```

- De igual forma, si intentamos ingresar un continente no existente, el sistema detecta un error y devuelve el correspondiente mensaje

```

==AÑADIR PAIS==
Dame el nombre del pais: Mexico
De cuanto es la poblacion de Mexico: 12200000
De cuanto es la superficie de Mexico (en km cuadrados): 300000000
En que continente esta ubicado Mexico: Francia
Contiente no válido

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadisticas
7) Salir
Que opcion desea elegir: 

```

- Si se ingresan los datos correspondientes, aparece una opción donde el usuario decide si agregar finalmente el elemento o no

```

==AÑADIR PAIS==
Dame el nombre del pais: Colombia
De cuanto es la poblacion de Colombia: 53500000
De cuanto es la superficie de Colombia (en km cuadrados): 1142000000
En que continente esta ubicado Colombia: America
==RESULTADO FINAL==
Pais: Colombia | Poblacion: 53500000 | Poblacion: 1142000000 | Continente: America
Desea guardar el pais?(si/no)

```

- En caso de 'no', el elemento no se guarda, y el código sigue operando

```

==AÑADIR PAIS==
Dame el nombre del pais: Mexico
De cuanto es la poblacion de Mexico: 12000000
De cuanto es la superficie de Mexico (en km cuadrados): 544444
En que continente esta ubicado Mexico: America
==RESULTADO FINAL==
Pais: Mexico | Poblacion: 12000000 | Poblacion: 544444 | Continente: America
Desea guardar el pais?(si/no)no
No se guardaron los datos

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadisticas
7) Salir
Que opcion desea elegir: 

```

- En caso de 'si', el elemento se escribe automáticamente en el archivo .csv

```

De cuanto es la superficie de Colombia (en km cuadrados): 1142000000
En que continente esta ubicado Colombia: America
==RESULTADO FINAL==
País: Colombia | Poblacion: 53500000 | Poblacion: 1142000000 | Continente: America
Desea guardar el pais?(si/no)si
Datos guardados correctamente!

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadísticas
7) Salir
Que opcion desea elegir: 

```

- Archivo .csv final

```

models > TpIntegrador > archivo.csv > data
1  Argentina,45376763,2780400,América
2  Japón,125800000,377975,Asia
3  Brasil,213993437,8515767,América
4  Alemania,344344,33333,Asia
5  Asas,2,2,Asia
6  Asa,2121212,22222222,Europa
7  Colombia,53500000,1142000000,America
8

```

BUSCAR PAIS

- Como vemos en el programa, si ingresamos un valor el cual no está registrado en el archivo .csv, muestra un mensaje de 'no coincidencias'

```

==BUSCAR PAIS==
Dime el nombre del pais a buscar: Hola
No hay resultados para: 'Hola'

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadísticas
7) Salir
Que opcion desea elegir: 

```

```

models > TpIntegrador > archivo.csv > data
1  Argentina,45376763,2780400,América
2  Japón,125800000,377975,Asia
3  Brasil,213993437,8515767,América
4  Alemania,344344,33333,Asia
5  Asas,2,2,Asia
6  Asa,2121212,22222222,Europa
7

```


- Si se ingresa un substring existente dentro de un string, el programa busca todos los elementos cuyo substring este dentro del nombre del país

```
==BUSCAR PAIS==
Dime el nombre del pais a buscar: A
==RESULTADOS==
1) Pais: Argentina | Poblacion: 45376763 | Superficie: 2780400 | Continente : América
2) Pais: Alemania | Poblacion: 344344 | Superficie: 33333 | Continente : Asia
3) Pais: Asas | Poblacion: 2 | Superficie: 2 | Continente : Asia
4) Pais: Asa | Poblacion: 2121212 | Superficie: 22222222 | Continente : Europa

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadísticas
7) Salir
Que opcion desea elegir: █
```

- Y por supuesto, se puede buscar el nombre completo del país

```
==BUSCAR PAIS==
Dime el nombre del pais a buscar: brasil
==RESULTADOS==
1) Pais: Brasil | Poblacion: 213993437 | Superficie: 8515767 | Continente : América

==OPCIONES==
1) Agregar pais
2) Actualizar datos pais
3) Buscar pais
4) Aplicar filtros
5) Aplicar Ordenes
6) Mostrar estadísticas
7) Salir
Que opcion desea elegir: █
```

REFLEXION FINAL, OBSTACULOS PRESENTADOS Y COMO SE RESOLVIERON

A lo largo de la escritura del código (no es una sorpresa) surgieron múltiples problemas al momento de ejecución, logística y tipeo. Mala indentacion, sintaxis errónea, bucles infinitos, entre otros. Sin embargo, hubo un problema que fue el principal obstáculo: “La lógica del código”

Coordinar el flujo de los datos, anticipar las diferentes condiciones y estructurar las reglas del algoritmo de manera ordenada presentó el mayor obstáculo durante el desarrollo.

La solución definitiva fue plasmar la estructura de forma visual. Diseñar un **diagrama de flujo** resultó fundamental para organizar las ideas, mapear con claridad cada camino lógico y desglosar el problema en pasos más pequeños y manejables. Esta herramienta no solo destrabó el avance del proyecto, sino que demostró que una

visualización grafica del programa es un pilar indispensable antes de escribir cualquier línea de código

ENLACES

- [Respositorio GitHub](#)
- [Video ExplicativoCodigo](#)

BIBLIOGRAFIA ÚTIL

- <https://docs.python.org/3/tutorial/datastructures.html>
- https://www.w3schools.com/python/python_conditions.asp
- <https://www.adobe.com/es/acrobat/resources/document-files/text-files/csv-file.html>
- <https://ellibrodepython.com/funciones-en-python>
- <https://tutorial.rekursospython.com/bucles/>